

10.2.8

EE25BTECH11002 - Achat Parth Kalpesh

October 11,2025

Question

The shortest distance from the point $(2,7)$ to the circle $x^2 + y^2 - 14x - 10y - 151 = 0$ is equal to 5

Solution

The general equation of a circle is

$$\mathbf{x}^T \mathbf{x} + 2\mathbf{u}^T \mathbf{x} + f = 0 \quad (1)$$

where

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} -7 \\ -5 \end{pmatrix}, \quad f = -151 \quad (2)$$

The center and radius are given by

$$\mathbf{C} = -\mathbf{u} = \begin{pmatrix} 7 \\ 5 \end{pmatrix} \quad (3)$$

$$r = \sqrt{\mathbf{u}^T \mathbf{u} - f} \quad (4)$$

$$r = 15 \quad (5)$$

Solution

Let the given point be $\mathbf{P}$

$$\mathbf{P} = \begin{pmatrix} 2 \\ 7 \end{pmatrix} \quad (6)$$

The distance between $\mathbf{P}$ and $\mathbf{C}$ is

$$\|\mathbf{P} - \mathbf{C}\| = \sqrt{(\mathbf{P} - \mathbf{C})^T (\mathbf{P} - \mathbf{C})} \quad (7)$$

$$= \sqrt{(2 - 7)^2 + (7 - 5)^2} \quad (8)$$

$$= \sqrt{25 + 4} = \sqrt{29} \quad (9)$$

Substituting (6) in (1)

$$\mathbf{P}^T \mathbf{P} + 2\mathbf{u}^T \mathbf{P} + f < 0 \quad (10)$$

Solution

The shortest distance of the point inside the circle is given as

$$d = r - \|\mathbf{P} - \mathbf{C}\| \quad (11)$$

Hence,

$$d = 15 - \sqrt{29} \quad (12)$$

Thus the shortest distance from the point (2,7) to the circle $x^2 + y^2 - 14x - 10y - 151 = 0$ is not equal to 5, it's $15 - \sqrt{29}$

```
#include <stdio.h>
#include <math.h>
#ifndef pi
#define pi acos(-1.0)
#endif

// Compute the center components
void compute_center(double u[2], double C[2]) {
    C[0] = -u[0];
    C[1] = -u[1];
}

// Compute the radius
double compute_radius(double u[2], double f) {
    return sqrt(u[0]*u[0] + u[1]*u[1] - f);
}
```

```
void generate_circle_points(double C[2], double r, double *x,
    double *y, int n) {
    double theta;
    for (int i = 0; i < n; i++) {
        double theta = 2.0 * pi * i / n;
        x[i] = C[0] + r * cos(theta);
        y[i] = C[1] + r * sin(theta);
    }
}

void compute_edge_point(double C[2], double P[2], double r,
    double edge[2]) {
    double dir[2];
    double mag = sqrt(pow(P[0]-C[0], 2) + pow(P[1]-C[1], 2));
    dir[0] = (P[0]-C[0]) / mag;
    dir[1] = (P[1]-C[1]) / mag;
    edge[0] = C[0] + r * dir[0];
    edge[1] = C[1] + r * dir[1];
}
```

Python Code

```
import numpy as np
import matplotlib.pyplot as plt
import ctypes

# Load shared library
lib = ctypes.CDLL('./formula.so')

# Declare argument and return types for each C function
lib.compute_center.argtypes = [ctypes.POINTER(ctypes.c_double),
                                ctypes.POINTER(ctypes.c_double)]
lib.compute_radius.argtypes = [ctypes.POINTER(ctypes.c_double),
                                ctypes.c_double]
lib.compute_radius.restype = ctypes.c_double
lib.generate_circle_points.argtypes = [
    ctypes.POINTER(ctypes.c_double), ctypes.c_double,
    ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.
        c_double),
    ctypes.c_int
]
```


Python Code

```
lib.compute_edge_point.argtypes = [  
    ctypes.POINTER(ctypes.c_double), ctypes.POINTER(ctypes.  
        c_double),  
    ctypes.c_double,  
    ctypes.POINTER(ctypes.c_double)  
]  
  
# --- Step 3: Set up input values ---  
u = (ctypes.c_double * 2)(-7, -5)  
f = ctypes.c_double(-151)  
P = np.array([2, 7], dtype=np.float64)  
  
# Compute center using C  
C = (ctypes.c_double * 2)()  
lib.compute_center(u, C)  
  
# Compute radius using C  
r = lib.compute_radius(u, f)
```

```
# Generate circle points using C
N = 400
x_arr = (ctypes.c_double * N)()
y_arr = (ctypes.c_double * N)()
lib.generate_circle_points(C, r, x_arr, y_arr, N)

# Convert circle arrays to NumPy for plotting
x_circ = np.array(list(x_arr))
y_circ = np.array(list(y_arr))

# Compute edge point using C
edge = (ctypes.c_double * 2)()
P_c = (ctypes.c_double * 2)(*P) # Convert NumPy -> C array
lib.compute_edge_point(C, P_c, r, edge)
```

```
# --- Step 4: Plot EXACTLY like your specified plot ---
plt.plot(x_circ, y_circ, label='Circle:  $x^2+y^2-14x-10y-151=0$ ',
         color='b')
plt.scatter(C[0], C[1], color='red')
plt.scatter(P[0], P[1], color='green')
plt.plot([C[0], P[0]], [C[1], P[1]], 'k--', label='Distance  $CP = 15 - \sqrt{29}$ ')
plt.text(C[0]+0.5, C[1]-1, f'C({C[0]:.0f},{C[1]:.0f})', fontsize=10)
plt.text(P[0]-1.5, P[1]+0.5, f'P({P[0]:.0f},{P[1]:.0f})',
         fontsize=10)
plt.text(P[0]+2.3, P[1], '$15 - \sqrt{29}$', fontsize=10)
# Plot radius in same direction
plt.plot([C[0], edge[0]], [C[1], edge[1]], 'r:', label='Radius = 15')
```

```
plt.gca().set_aspect('equal', adjustable='box')
plt.grid(True, linestyle='--', alpha=0.5)
plt.xlabel('X - axis')
plt.ylabel('Y - axis')
plt.title('Shortest Distance from Point (2,7) to Circle')
plt.legend(loc='upper right')
plt.show()
```

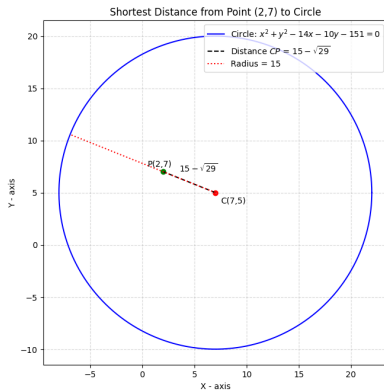


Figure: Shortest Distance from Point (2,7) to Circle